

```

163.         print('(B)all, ', end='')
164.         bext.fg('cyan')
165.         print('(C)lub, ', end='')
166.         bext.fg('purple')
167.         print('(S)pade, ', end='')
168.     bext.fg('white')
169.     print('or QUIT:')
170.     response = input('> ').upper()
171.     if response == 'QUIT':
172.         print('Thanks for playing!')
173.         sys.exit()
174.     if displayMode == COLOR_MODE and response in tuple('RGBYCP'):
175.         # Return a tile type number based on the response:
176.         return {'R': 0, 'G': 1, 'B': 2,
177.                'Y': 3, 'C': 4, 'P': 5}[response]
178.     if displayMode == SHAPE_MODE and response in tuple('HTDBCS'):
179.         # Return a tile type number based on the response:
180.         return {'H': 0, 'T': 1, 'D': 2,
181.                'B': 3, 'C': 4, 'S': 5}[response]
182.
183.
184. def changeTile(tileType, board, x, y, charToChange=None):
185.     """Change the color/shape of a tile using the recursive flood fill
186.     algorithm."""
187.     if x == 0 and y == 0:
188.         charToChange = board[(x, y)]
189.         if tileType == charToChange:
190.             return # Base Case: Already is the same tile.
191.
192.     board[(x, y)] = tileType
193.
194.     if x > 0 and board[(x - 1, y)] == charToChange:
195.         # Recursive Case: Change the left neighbor's tile:
196.         changeTile(tileType, board, x - 1, y, charToChange)
197.     if y > 0 and board[(x, y - 1)] == charToChange:
198.         # Recursive Case: Change the top neighbor's tile:
199.         changeTile(tileType, board, x, y - 1, charToChange)
200.     if x < BOARD_WIDTH - 1 and board[(x + 1, y)] == charToChange:
201.         # Recursive Case: Change the right neighbor's tile:
202.         changeTile(tileType, board, x + 1, y, charToChange)
203.     if y < BOARD_HEIGHT - 1 and board[(x, y + 1)] == charToChange:
204.         # Recursive Case: Change the bottom neighbor's tile:
205.         changeTile(tileType, board, x, y + 1, charToChange)
206.
207.
208. def hasWon(board):
209.     """Return True if the entire board is one color/shape."""
210.     tile = board[(0, 0)]
211.
212.     for x in range(BOARD_WIDTH):
213.         for y in range(BOARD_HEIGHT):
214.             if board[(x, y)] != tile:
215.                 return False
216.     return True
217.

```